CHAPTER 03

Classes & Object-Oriented Programming

C# Programming

Classes • Static Methods • Overloading
Access Modifiers • Inheritance • Interfaces

A CLASS BLUEPRINT

```
public class Meter
{
    public string SerialNumber { get; set; }

    public double Reading { get; set; }


    public void DisplayReading()
    {
        Console.WriteLine(Reading);
    }
}
```

LEARN • UNDERSTAND • BUILD

What we'll learn

Build from class fundamentals to inheritance, interfaces, and a practical Meter model.

01 **Classes**
Blueprints, objects, class features, and core components.

02 **Static methods**
Class-level methods and calling methods through a class name.

03 **Polymorphism**
Constructor overloading and method overloading.

04 **Access modifiers**
Control visibility and protect class data.

05 **Inheritance & interfaces**
Reuse behaviour and define shared contracts.

06 **Final assignment**
Design a Meter class hierarchy and interface.

What is a class?

A class is a user-defined blueprint or template used to create objects in C#.

Blueprint

A class describes what an object will contain and what it can do.

Related data

Group data such as fields and properties into one structured unit.

Related behaviour

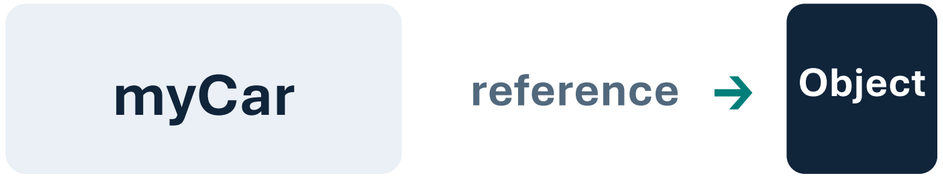
Group actions such as methods into the same class.



OOP characteristic: reference types

Classes are reference types. A variable stores a reference to an object rather than the object's data itself.

VARIABLE



The object's data lives on the heap.

Key idea

When an object is created, a variable holds a reference to the memory location where the object's data lives.

OOP characteristic: encapsulation

Encapsulation protects the internal state of an object by controlling access to its data.

Private data

Use private fields to prevent unrelated code from changing internal data directly.

Controlled access

Use public properties or methods to decide how data can be read or updated.

Encapsulation = protect data + provide controlled access

OOP characteristic: polymorphism

Polymorphism allows different classes to respond in their own way to the same method call.

virtual

A base-class method can be marked virtual so a derived class is allowed to replace its behaviour.

override

A derived class uses override to provide its own implementation of the inherited virtual method.

The same method call can produce different behaviour depending on the object.

OOP characteristic: inheritance

C# classes support single inheritance: a class can inherit from one base class.

Vehicle • base class ↓ inherited by **Truck • derived class**

Base class

Provides members that can be inherited.

Derived class

Inherits accessible members and can add its own members.

Core components of a C# class

A well-defined class typically combines data, controlled access, initialization, and behaviour.

Fields

Variables declared directly inside a class to hold data.

Properties

Provide a flexible way to read, write, or compute field values.

Constructor

A special method called automatically when an object is created; used to initialize its data.

Methods

Actions or operations that the class can perform.

How the Car example is organised

The example uses a field, properties, a constructor, and a method inside one class.

- Field: private string model;
- Auto-property: public string Color { get; set; }
- Property: Model reads and updates the private model field.
- Constructor: initializes Model and Color when a Car is created.
- Method: Drive() displays what the car is doing.

Car class: field and auto-property

Start the class by declaring a private field and an auto-implemented property.

C#

```
using System;

public class Car
{
    // Field
    private string model;

    // Auto-implemented property
    public string Color { get; set; }
```

Car class: Model property

The Model property provides controlled access to the private model field.

C#

```
        // Property with explicit get and set
        public string Model
        {
            get
            {
                return model;
            }
            set
            {
                model = value;
            }
        }
```

Car class: constructor

The constructor runs automatically when a new Car object is created.

C#

```
// Constructor
public Car(string model, string color)
{
    Model = model;
    Color = color;
}

// The class continues with its methods...
```

The this keyword

Use this to refer to the current object inside a class.

C#

```
01  public class Car
02  {
03      private string model;
04
05      public Car(string model)
06      {
07          this.model = model;
08      }
09  }
```

WHAT DOES IT MEAN?

this refers to the current object (the instance of the class).

IN THIS EXAMPLE

`this.model` = field

`model` = parameter

WHY USE IT?

It removes confusion when a field and parameter have the same name.

NOTE • In C#, this is a keyword (not an operator).

Car class: Drive() method

Drive() represents an action that the Car object can perform.

C#

```
        // Method (behaviour)
        public void Drive()
        {
            Console.WriteLine(
                $"The {Color} {Model} is driving down the road.");
        }
    }
```

Create a Car object

Use the new keyword to create an object, then call its method through the object variable.

C#

```
class Program
{
    static void Main()
    {
        // Create an object using new
        Car myCar = new Car("Tesla Model 3", "Red");

        // Call a method on the object
        myCar.Drive();
    }
}
```

Car example: output and key terms

The method call displays the car’s colour and model in the console.

CONSOLE OUTPUT

```
The Red Tesla Model 3
is driving down the road.
```

Car

The class / blueprint.

myCar

The object variable created with new.

What is a static method?

A static method belongs to the class itself rather than to a specific object.

Belongs to class

Declare it using the static keyword inside a class.

Call by class name

Use ClassName.MethodName() to call it directly.

No object required

You do not need to create an instance with new first.

```
Calculator.Add(5, 10)     •     Calculator.Square(4)
```

Static Calculator: Add()

The Add() method receives two integer parameters and returns their sum.

C#

```
using System;
namespace StaticMethodDemo
{
    public class Calculator
    {
        // Static method to add two numbers
        public static int Add(int a, int b)
        {
            return a + b;
        }
```

Static Calculator: Square()

The Square() method receives a number and returns the number multiplied by itself.

C#

```
        // Static method to calculate a square
        public static double Square(double number)
        {
            return number * number;
        }
    }
}
```

Call static methods from Main()

Call each method through the Calculator class name and store the returned values.

C#

```
class Program
{
    static void Main(string[] args)
    {
        // Call with the class name
        int sum = Calculator.Add(5, 10);
        double squaredValue = Calculator.Square(4);

        // No new Calculator() is needed
        Console.WriteLine($"The sum is: {sum}");
        Console.WriteLine($"The square is: {squaredValue}");
    }
}
```

Static method: output

The methods return values that are stored and then printed.

Sum

Calculator.Add(5, 10)

Output: 15

Square

Calculator.Square(4)

Output: 16

No Calculator object is created in this example.

Polymorphism: overloading

This chapter introduces polymorphism through constructor overloading and method overloading.

Constructor overloading

A class can have multiple constructors with the same name but different parameter lists. Objects can be initialized in different ways.

Method overloading

A class can have multiple methods with the same name but different parameter lists. The arguments determine which version is called.

What makes a constructor signature unique?

Each constructor must have a unique parameter signature so the compiler can distinguish it.

Number

For example: a constructor with no parameters and another with two parameters.

Types

For example: one constructor accepts an int while another accepts a string.

Order

For example: (string, int) and (int, string).

Product class: properties

The Product class stores a name, price, and stock value using properties.

C#

```
using System;
public class Product
{
    public string Name { get; set; }
    public double Price { get; set; }
    public int Stock { get; set; }
```

Product class: default constructor

The parameterless constructor initializes the object with default product values.

C#

```
01        // Default constructor (no parameters)
02        public Product()
03        {
04            Name = "Generic Product";
05            Price = 0.0;
06            Stock = 0;
07        }
```

Product class: two-parameter constructor

This overloaded constructor accepts a name and price, and sets stock to 10 by default.

C#

```
// Overloaded constructor (two parameters)
public Product(string name, double price)
{
    Name = name;
    Price = price;
    Stock = 10;
}
```

Product class: three-parameter constructor

This constructor accepts all three values: name, price, and stock.

C#

```
01        // Overloaded constructor (three parameters)
02        public Product(string name, double price, int stock)
03        {
04            Name = name;
05            Price = price;
06            Stock = stock;
07        }
```

Product class: DisplayInfo()

DisplayInfo() prints the product name, price, and stock in one formatted line.

C#

```
        public void DisplayInfo()
        {
            Console.WriteLine(
                $"Product: {Name} | Price: ${Price} | Stock: {Stock}");
        }
    }
```

Create Product objects: first two

The compiler selects the matching constructor from the arguments supplied to new Product().

C#

```
class Program
{
    static void Main()
    {
        // Calls the default constructor
        Product p1 = new Product();

        // Calls the two-parameter constructor
        Product p2 = new Product("Laptop", 999.99);
```

Create Product objects: third constructor

The three-parameter constructor allows the stock value to be supplied when the object is created.

C#

```
            // Calls the three-parameter constructor
            Product p3 = new Product("Smartphone", 699.99, 50);


            p1.DisplayInfo();
            p2.DisplayInfo();
            p3.DisplayInfo();
        }
    }
```

Constructor overloading: what to notice

All three objects are Products, but each can be initialized with a different set of values.

p1

Product()
Generic Product • 0.0 • 0

p2

Product("Laptop", 999.99)
Stock defaults to 10.

p3

Product("Smartphone", 699.99, 50)
Stock is supplied as 50.

What is method overloading?

A class can contain methods with the same name, as long as their parameter signatures differ.

The methods must differ by at least one of these:

- Number of parameters
- Types of parameters
- Order of parameters

Overloading by number of parameters

These Add() methods share the same name but accept different numbers of integer parameters.

C#

```
// Two integer parameters
public int Add(int a, int b)
{
    return a + b;
}

// Three integer parameters
public int Add(int a, int b, int c)
{
    return a + b + c;
}
```

Overloading by parameter type

The same Add() method name can accept different numeric types.

C#

```
// Two double parameters
public double Add(double a, double b)
{
    return a + b;
}
```

Overloading by parameter order

DisplayLog() has the same name and types, but the order of parameters is different.

C#

```
01  public void DisplayLog(string message, int code)
02  {
03      Console.WriteLine($"[{code}] {message}");
04  }

05  public void DisplayLog(int code, string message)
06  {
07      Console.WriteLine($"Code: {code} -> {message}");
08  }
```

Call the overloaded Add() methods

The compiler chooses an overload by matching the number and type of arguments.

C#

```
Calculator calc = new Calculator();

Console.WriteLine(calc.Add(5, 10));
// Selects Add(int, int)

Console.WriteLine(calc.Add(5, 10, 20));
// Selects Add(int, int, int)

Console.WriteLine(calc.Add(5.5, 4.5));
// Selects Add(double, double)
```

Call the overloaded DisplayLog() methods

The argument order determines which DisplayLog() version is called.

C#

```
Calculator calc = new Calculator();

calc.DisplayLog("Error occurred", 404);
// Calls DisplayLog(string, int)

calc.DisplayLog(404, "Error occurred");
// Calls DisplayLog(int, string)
```

Method overloading: example output

Each method call selects the matching overload and produces its own result.

CONSOLE OUTPUT

```
15
35
10
[404] Error occurred
Code: 404 -> Error occurred
```

Access modifiers: public and private

Access modifiers determine where a class or class member can be accessed.

public

Accessible from any code that can access the containing type.

private

Accessible only inside the containing type.

Access modifiers: protected and internal

These modifiers control access through inheritance or assembly boundaries.

protected

Accessible inside the containing type and its derived classes.

internal

Accessible within the same assembly.

Combined access modifiers

C# also provides combined modifiers for more specific access rules.

protected internal

Accessible within the same assembly OR from derived types.

private protected

Accessible within the containing type or derived types in the same assembly.

Access modifiers: quick comparison

Use the modifier that provides the minimum access needed by a member.

public	Any code that can access the type
private	The containing type only
protected	The containing type and derived classes
internal	The same assembly
protected internal	Same assembly OR derived types
private protected	Containing type or derived types in same assembly

Encapsulation example: protect Meter data

Keep the reading in a private field and expose it through controlled properties and methods.

C#

```
public class MeterAccount
{
    private double currentReading;

    public string SerialNumber { get; private set; }
```

MeterAccount: read-only reading property

The public property exposes the current reading through a getter, without a public setter.

C#

```
public double CurrentReading
{
    get
    {
        return currentReading;
    }
}
```

MeterAccount: constructor and update method

The constructor sets the serial number. UpdateReading() changes the private reading field.

C#

```
        public MeterAccount(string serialNumber)
        {
            SerialNumber = serialNumber;
        }

        public void UpdateReading(double newReading)
        {
            currentReading = newReading;
        }
    }
```

What is inheritance?

Inheritance lets a new class reuse data and behaviour from an existing class.

Base class

The existing class whose accessible members can be inherited.

Derived class

The new class that inherits from a base class and can add its own members.

Single inheritance

A C# class can inherit from one base class.

```
class Vehicle { ... }
```

↓

```
class Truck : Vehicle { ... }
```

Vehicle base class: property and constructor

The base class stores the brand and initializes it through a constructor.

C#

```
public class Vehicle
{
    public string Brand { get; set; }

    public Vehicle(string brand)
    {
        Brand = brand;
    }
```

Vehicle base class: Start() method

Start() is a method inherited by classes derived from Vehicle.

C#

```
        public void Start()
        {
            Console.WriteLine($"{Brand} is starting.");
        }
    }
```

Truck derived class

Truck inherits from Vehicle and adds its own LoadCapacity property.

C#

```
01  public class Truck : Vehicle
02  {
03      public int LoadCapacity { get; set; }
```

Truck constructor: base(brand)

The base(brand) call passes the brand value to the Vehicle constructor.

C#

```
        public Truck(string brand, int loadCapacity) : base(brand)
        {
            LoadCapacity = loadCapacity;
        }
    }
```

Method overriding: virtual

A base-class method can be marked virtual to allow a derived class to replace its implementation.

C#

```
public class Vehicle
{
    public virtual void Start()
    {
        Console.WriteLine("Vehicle starts.");
    }
}
```

Method overriding: override

The derived ElectricVehicle class overrides Start() and supplies its own behaviour.

C#

```
public class ElectricVehicle : Vehicle
{
    public override void Start()
    {
        Console.WriteLine(
            "Electric vehicle starts silently.");
    }
}
```

Run the overridden method

The object is an ElectricVehicle, so the overridden Start() method is executed.

C#

```
01  class Program
02  {
03      static void Main()
04      {
05          Vehicle myVehicle = new ElectricVehicle();
06          myVehicle.Start();
07      }
08  }
```

What is an interface?

An interface defines a contract: it declares members that an implementing class must provide.

A contract

An interface describes which members a type must provide.

Implemented by classes

A class uses a colon to implement an interface and supplies its required members.

Shared behaviour

Different classes can provide their own implementation of the same contract.

Interface syntax: IReadable

This interface declares a method signature. It specifies what a class must provide.

C#

```
01  public interface IReadable
02  {
03      double ReadValue();
04  }
```

Implement IReadable: TemperatureSensor

TemperatureSensor implements IReadable by providing the required ReadValue() method.

C#

```
public class TemperatureSensor : IReadable
{
    public double ReadValue()
    {
        return 23.5;
    }
}
```

Implement IReadable: PressureSensor

PressureSensor implements the same contract but returns its own reading.

C#

```
public class PressureSensor : IReadable
{
    public double ReadValue()
    {
        return 101.3;
    }
}
```

Inheritance vs. interfaces

Both support reuse, but they describe different relationships.

Inheritance: “is a”

A derived class inherits accessible members and can extend or override base-class behaviour. A C# class can inherit from one base class.

Interface: “can do”

A class implements the members declared by an interface. Multiple classes can implement the same contract in their own way.

Final assignment: Meter class model

Apply classes, properties, access modifiers, constructors, inheritance, and interfaces together.

YOUR TASK

Build a Meter model

Create a base Meter class with ElectricMeter and GasMeter subclasses. Implement a shared interface for common meter behaviour.

Meter assignment: class requirements

Use the class features and access control concepts from this chapter.

- Create a Meter base class.
- Add properties such as SerialNumber and Reading.
- Use suitable access modifiers to protect internal state.
- Use constructors to initialize meter data.

Meter assignment: inheritance and interface

Complete the class hierarchy and demonstrate the shared behaviour.

- Create ElectricMeter and GasMeter as subclasses of Meter.
- Define an interface for common meter behaviour.
- Implement the interface in both derived classes.
- Create objects and demonstrate the properties and methods.

Use idiomatic C# syntax and demonstrate object creation and method calls.